TCS IPA 35-Mark Java Practice Master Handbook

Corrected Full Edition - 15 problems, every program complete and tested

What was fixed in this edition

- Problems 2, 3 and 5: the Java listing was cut off in the middle of the POJO class (it stopped after a few getters). The complete class, with every getter and setter, is now included.

- Problem 2: the sample output in the original handbook printed laptops 104, 101, 103, which is NOT descending ID order. The correct output is 104, 103, 101. The program was right; the printed sample was wrong.

- Problems 6 to 15 were listed in the pattern table but never written out. They are now complete, with a problem statement, sample I/O and full code.

- Every program was compiled and run with Java 21 on the sample input AND on an edge case (no match found). The outputs printed in this PDF are the real program outputs, not typed by hand.

- Second-highest logic (Problems 8, 10, 12, 15) now uses the 'first value strictly below the maximum' rule, so duplicate values cannot give a wrong answer.

CRITICAL EXAM HAZARD - Scanner buffer flushing: after sc.nextInt() or sc.nextDouble(), the newline stays in the buffer. Before reading a String with sc.nextLine() you must call sc.nextLine() once to clear it. Every listing here shows exactly where.

Exam template (always in this order)

```
import java.util.*;

public class Solution {
    // 1. static business methods
    // 2. public static void main(String[] args)  -> read input, call methods, print
}
// 3. POJO class: private fields, constructor, getters + setters
```

Reusable patterns

- MAX / MIN with filter: X best = null; if (matches) { if (best == null || v.get() > best.get()) best = v; }

- AVERAGE: return count > 0 ? sum / count : 0; (integer division, guard against zero)

- SORT: list.sort((a, b) -> a.getX() - b.getX()); ascending. Swap a and b for descending. Use Double.compare for doubles.

- SECOND HIGHEST: sort descending, then return the first item whose value < list.get(0)'s value; otherwise null.

- STRINGS: always compare with equalsIgnoreCase(), never ==.

- RETURN null / 0 when nothing matches, and print the exact message the question asks for.

The 15 problems

#	Domain	Method 1	Method 2
1	Fruits	findMaximumPriceByRating	-
2	Laptop	countOfLaptopsByBrand	searchLaptopByOsType
3	Player	findPlayerWithLowestRuns	findAverageRunsByMatchType
4	Medicine	getMedicineByDisease	-
5	Course	findAvgFeeByAdmin	sortCourseByFee
6	Phone	findPriceForGivenBrand	getPhoneByOsAndMinPrice
7	Book	findBookWithMaxPagesByAuthor	countBooksAbovePrice
8	Employee	countEmployeesByBranch	findEmployeeWithSecondHighestSalary
9	Movie	findAvgBudgetByGenre	getMoviesByRatingSortedByBudget
10	AutonomousCar	updateGradeByEnvironment	findCarWithSecondHighestPassRate
11	Inventory	filterItemsNeedingRefill	countByThresholdCategory
12	Hotel	noOfRoomsBookedInGivenMonth	searchHotelWithSecondHighestBill
13	NavalVessel	findAvgVoyagesByVesselClass	findVesselWithHighestVoyagesByMode
14	Sim	transferBalance	changeCircle
15	Footwear	countFootwearByType	getSecondHighestPriceByName

Problems 6 to 15 follow the standard TCS IPA pattern, but exact wording differs between exam papers. Where the table was vague (for example the category rules in Problem 11 or the rating filter in Problem 9) the assumption is stated in the problem. Always follow the wording of YOUR question paper.

Problem 01: Fruits Management System - Problem Statement and I/O

1. Entity attributes (class Fruits)

Class Fruits with private attributes: int fruitId, String fruitName, int price, int rating. Write a parameterized constructor and getters/setters in the same sequence.

2. Static business methods (class Solution)

`findMaximumPriceByRating(Fruits[] arr, int targetRating)`

Returns the maximum-priced Fruits object whose rating is strictly greater than targetRating. If no fruit qualifies, return null.

3. Main execution rules

Read 4 Fruits objects, then targetRating. Call the method. If the result is non-null print fruitId, otherwise print "No such Fruit".

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - nobody has rating above 5

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
555 Apple 200 5 777 Orange 150 4 333 Banana 100 4 888 Avocado 250 4 3	888	555 Apple 200 5 777 Orange 150 4 333 Banana 100 4 888 Avocado 250 4 5	No such Fruit

Explanation: All four fruits have rating > 3. Avocado has the highest price (250), so its ID 888 is printed.

Exam strategy: Start with Fruits maxFruit = null; update when maxFruit == null || f.getPrice() > maxFruit.getPrice().

Problem 01: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Fruits POJO. Copy as one file, Solution.java.

```
import java.util.Scanner;

public class Solution {

    // 1. STATIC BUSINESS LOGIC FIRST
    public static Fruits findMaximumPriceByRating(Fruits[] arr, int targetRating) {
        Fruits maxFruit = null;
        for (Fruits f : arr) {
            if (f.getRating() > targetRating) {
                if (maxFruit == null || f.getPrice() > maxFruit.getPrice()) {
                    maxFruit = f;
                }
            }
        }
        return maxFruit;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Fruits[] fruits = new Fruits[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();                    // flush newline buffer
            String name = sc.nextLine();
            int price = sc.nextInt();
            int rating = sc.nextInt();
            sc.nextLine();                    // flush newline buffer
            fruits[i] = new Fruits(id, name, price, rating);
        }
        int targetRating = sc.nextInt();

        Fruits res = findMaximumPriceByRating(fruits, targetRating);
        if (res != null) {
            System.out.println(res.getFruitId());
        } else {
            System.out.println("No such Fruit");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Fruits {
    private int fruitId;
    private String fruitName;
    private int price;
    private int rating;

    public Fruits(int fruitId, String fruitName, int price, int rating) {
        this.fruitId = fruitId;
        this.fruitName = fruitName;
        this.price = price;
        this.rating = rating;
    }

    public int getFruitId() { return fruitId; }
    public void setFruitId(int fruitId) { this.fruitId = fruitId; }
    public String getFruitName() { return fruitName; }
    public void setFruitName(String fruitName) { this.fruitName = fruitName; }
    public int getPrice() { return price; }
    public void setPrice(int price) { this.price = price; }
    public int getRating() { return rating; }
    public void setRating(int rating) { this.rating = rating; }
}
```

Problem 02: Laptop Management - Problem Statement and I/O

1. Entity attributes (class Laptop)

Class Laptop with private attributes: int laptopId, String brand, String osType, double price, int rating. Constructor plus getters/setters.

2. Static business methods (class Solution)

`countOfLaptopsByBrand(Laptop[] arr, String brand)`
Returns the count of laptops matching brand (case-insensitive) with rating > 3. If none, return 0.

`searchLaptopByOsType(Laptop[] arr, String osType)`
Returns an array of laptops matching osType (case-insensitive), sorted by laptopId in DESCENDING order. If none, return null.

3. Main execution rules

Read 4 laptops, then searchBrand and searchOs. Print the count (or "The given brand is not available"). Then print each sorted laptop's ID and price on separate lines (or "The given OS is not available").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - brand and OS not present

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
101 Dell Windows 55000 4 102 Dell Ubuntu 45000 3 103 HP Windows 62000 5 104 Dell Windows 72000 5 Dell Windows	2 104 72000.0 103 62000.0 101 55000.0	101 Dell Windows 55000 4 102 Dell Ubuntu 45000 3 103 HP Windows 62000 5 104 Dell Windows 72000 5 Lenovo Linux	The given brand is not available The given OS is not available

Explanation: Two Dell laptops have rating > 3 (101 and 104), so the count is 2. Windows laptops are 101, 103 and 104; in DESCENDING ID order they print as 104, 103, 101 (each ID followed by its price). NOTE: the original handbook showed 104, 101, 103, which is not descending order - that sample output was wrong.

Exam strategy: Sort descending with list.sort((a, b) -> b.getLaptopId() - a.getLaptopId());

Problem 02: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Laptop POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int countOfLaptopsByBrand(Laptop[] arr, String brand) {
        int count = 0;
        for (Laptop lp : arr) {
            if (lp.getBrand().equalsIgnoreCase(brand) && lp.getRating() > 3) {
                count++;
            }
        }
        return count;
    }

    public static Laptop[] searchLaptopByOsType(Laptop[] arr, String osType) {
        List<Laptop> list = new ArrayList<>();
        for (Laptop lp : arr) {
            if (lp.getOsType().equalsIgnoreCase(osType)) {
                list.add(lp);
            }
        }
        if (list.isEmpty()) return null;
        list.sort((a, b) -> b.getLaptopId() - a.getLaptopId());   // descending
        return list.toArray(new Laptop[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Laptop[] laptops = new Laptop[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String brand = sc.nextLine();
            String os = sc.nextLine();
            double price = sc.nextDouble();
            int rating = sc.nextInt();
            sc.nextLine();
            laptops[i] = new Laptop(id, brand, os, price, rating);
        }
        String searchBrand = sc.nextLine();
        String searchOs = sc.nextLine();

        int count = countOfLaptopsByBrand(laptops, searchBrand);
        if (count > 0) System.out.println(count);
        else System.out.println("The given brand is not available");

        Laptop[] result = searchLaptopByOsType(laptops, searchOs);
        if (result != null) {
            for (Laptop lp : result) {
                System.out.println(lp.getLaptopId());
                System.out.println(lp.getPrice());
            }
        } else {
            System.out.println("The given OS is not available");
        }
        sc.close();
    }
}
// 3. ENTITY POJO CLASS
class Laptop {
    private int laptopId;
    private String brand;
    private String osType;
    private double price;
    private int rating;

    public Laptop(int laptopId, String brand, String osType, double price, int rating) {
        this.laptopId = laptopId;
        this.brand = brand;
        this.osType = osType;
        this.price = price;
        this.rating = rating;
    }

    public int getLaptopId() { return laptopId; }
    public void setLaptopId(int laptopId) { this.laptopId = laptopId; }
    public String getBrand() { return brand; }
```

```
    public void setBrand(String brand) { this.brand = brand; }
    public String getOsType() { return osType; }
    public void setOsType(String osType) { this.osType = osType; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
    public int getRating() { return rating; }
    public void setRating(int rating) { this.rating = rating; }
}
```

Problem 03: Sports Analytics (Player Runs) - Problem Statement and I/O

1. Entity attributes (class Player)

Class Player with private attributes: int playerId, String playerName, int runs, String playerType, String matchType. Constructor plus getters/setters.

2. Static business methods (class Solution)

`findPlayerWithLowestRuns(Player[] arr, String playerType)`
Returns the Player with the lowest runs whose playerType matches (case-insensitive). If none, return null.

`findAverageRunsByMatchType(Player[] arr, String matchType)`
Returns the INTEGER average runs for the matching matchType. If none, return 0.

3. Main execution rules

Read 4 players, then searchPType and searchMType. Print the lowest player's ID and name on separate lines (or "No such player"). Then print the average runs (or "No such match type").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - no Allrounder, no Test matches

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
11 Sachin 18000 Batsman ODI 12 Zaheer 1200 Bowler ODI 13 Virat 13000 Batsman ODI 14 Dhoni 10500 Batsman T20 Batsman ODI	14 Dhoni 10733	11 Sachin 18000 Batsman ODI 12 Zaheer 1200 Bowler ODI 13 Virat 13000 Batsman ODI 14 Dhoni 10500 Batsman T20 Allrounder Test	No such player No such match type

Explanation: Lowest-run Batsman is Dhoni (10500). Average for ODI = (18000 + 1200 + 13000) / 3 = 10733 (integer division).

Exam strategy: Division guard: return count > 0 ? (totalRuns / count) : 0; to avoid ArithmeticException.

Problem 03: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Player POJO. Copy as one file, Solution.java.

```
import java.util.Scanner;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static Player findPlayerWithLowestRuns(Player[] arr, String playerType) {
        Player minPlayer = null;
        for (Player p : arr) {
            if (p.getPlayerType().equalsIgnoreCase(playerType)) {
                if (minPlayer == null || p.getRuns() < minPlayer.getRuns()) {
                    minPlayer = p;
                }
            }
        }
        return minPlayer;
    }

    public static int findAverageRunsByMatchType(Player[] arr, String matchType) {
        int totalRuns = 0;
        int count = 0;
        for (Player p : arr) {
            if (p.getMatchType().equalsIgnoreCase(matchType)) {
                totalRuns += p.getRuns();
                count++;
            }
        }
        return count > 0 ? (totalRuns / count) : 0;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Player[] players = new Player[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            int runs = sc.nextInt();
            sc.nextLine();
            String pType = sc.nextLine();
            String mType = sc.nextLine();
            players[i] = new Player(id, name, runs, pType, mType);
        }
        String searchPType = sc.nextLine();
        String searchMType = sc.nextLine();

        Player lowestPlayer = findPlayerWithLowestRuns(players, searchPType);
        if (lowestPlayer != null) {
            System.out.println(lowestPlayer.getPlayerId());
            System.out.println(lowestPlayer.getPlayerName());
        } else {
            System.out.println("No such player");
        }

        int avgRuns = findAverageRunsByMatchType(players, searchMType);
        if (avgRuns > 0) {
            System.out.println(avgRuns);
        } else {
            System.out.println("No such match type");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Player {
    private int playerId;
    private String playerName;
    private int runs;
    private String playerType;
    private String matchType;

    public Player(int playerId, String playerName, int runs, String playerType, String matchType) {
        this.playerId = playerId;
        this.playerName = playerName;
        this.runs = runs;
        this.playerType = playerType;
        this.matchType = matchType;
    }
```

```
    public int getPlayerId() { return playerId; }
    public void setPlayerId(int playerId) { this.playerId = playerId; }
    public String getPlayerName() { return playerName; }
    public void setPlayerName(String playerName) { this.playerName = playerName; }
    public int getRuns() { return runs; }
    public void setRuns(int runs) { this.runs = runs; }
    public String getPlayerType() { return playerType; }
    public void setPlayerType(String playerType) { this.playerType = playerType; }
    public String getMatchType() { return matchType; }
    public void setMatchType(String matchType) { this.matchType = matchType; }
}
```

Problem 04: Medicine Inventory - Problem Statement and I/O

1. Entity attributes (class Medicine)

Class Medicine with private attributes: String medicineName, String batchNo, String disease, int price. Constructor plus getters/setters.

2. Static business methods (class Solution)

`getMedicineByDisease(Medicine[] arr, String disease)`

Returns an int[] of prices of medicines matching the disease (case-insensitive), sorted ASCENDING. If no medicine matches, return null.

3. Main execution rules

Read 4 medicines, then the target disease. Print each price on a new line. If the result is null print "No Medicine found".

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - disease not present

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
Dolo650 B12 Fever 30 Paracetamol B14 Fever 20 Cetrizine B19 Allergy 45 Azithral B22 Fever 120 Fever	20 30 120	Dolo650 B12 Fever 30 Paracetamol B14 Fever 20 Cetrizine B19 Allergy 45 Azithral B22 Fever 120 Cold	No Medicine found

Explanation: Fever medicines are Paracetamol (20), Dolo650 (30) and Azithral (120), printed in ascending order of price.

Exam strategy: Collect prices into a List<Integer>, call Collections.sort(list), then copy into a new int[].

Problem 04: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Medicine POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS LOGIC FIRST
    public static int[] getMedicineByDisease(Medicine[] arr, String disease) {
        List<Integer> list = new ArrayList<>();
        for (Medicine m : arr) {
            if (m.getDisease().equalsIgnoreCase(disease)) {
                list.add(m.getPrice());
            }
        }
        if (list.isEmpty()) return null;
        Collections.sort(list);                      // ascending
        int[] result = new int[list.size()];
        for (int i = 0; i < list.size(); i++) {
            result[i] = list.get(i);
        }
        return result;
    }
    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Medicine[] meds = new Medicine[4];
        for (int i = 0; i < 4; i++) {
            String name = sc.nextLine();
            String batch = sc.nextLine();
            String disease = sc.nextLine();
            int price = sc.nextInt();
            sc.nextLine();                           // flush newline buffer
            meds[i] = new Medicine(name, batch, disease, price);
        }
        String targetDisease = sc.nextLine();

        int[] prices = getMedicineByDisease(meds, targetDisease);
        if (prices != null) {
            for (int p : prices) {
                System.out.println(p);
            }
        } else {
            System.out.println("No Medicine found");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Medicine {
    private String medicineName;
    private String batchNo;
    private String disease;
    private int price;

    public Medicine(String medicineName, String batchNo, String disease, int price) {
        this.medicineName = medicineName;
        this.batchNo = batchNo;
        this.disease = disease;
        this.price = price;
    }
    public String getMedicineName() { return medicineName; }
    public void setMedicineName(String medicineName) { this.medicineName = medicineName; }
    public String getBatchNo() { return batchNo; }
    public void setBatchNo(String batchNo) { this.batchNo = batchNo; }
    public String getDisease() { return disease; }
    public void setDisease(String disease) { this.disease = disease; }
    public int getPrice() { return price; }
    public void setPrice(int price) { this.price = price; }
}
```

Problem 05: Course Administration - Problem Statement and I/O

1. Entity attributes (class Course)

Class Course with private attributes: int courseId, String courseName, String courseAdmin, int fee, int rating. Constructor plus getters/setters.

2. Static business methods (class Solution)

`findAvgFeeByAdmin(Course[] arr, String admin)`
Returns the INTEGER average fee of courses by courseAdmin (case-insensitive). If none, return 0.

`sortCourseByFee(Course[] arr, int ratingThreshold)`
Returns an array of courses with rating <= ratingThreshold, sorted by fee ASCENDING. If none, return null.

3. Main execution rules

Read 4 courses, then admin name and rating threshold. Print the average fee (or "No Such Courses"). Then print matching course names one per line (or "No certificate found").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - unknown admin, rating threshold 1

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
101 Java John 15000 4 102 Python John 12000 5 103 SQL Alex 8000 3 104 AWS John 20000 3 John 4	15666 SQL Java AWS	101 Java John 15000 4 102 Python John 12000 5 103 SQL Alex 8000 3 104 AWS John 20000 3 Zoe 1	No Such Courses No certificate found

Explanation: John's average fee = (15000 + 12000 + 20000) / 3 = 15666. Courses with rating <= 4 sorted by fee: SQL (8000), Java (15000), AWS (20000).

Exam strategy: Ascending sort: list.sort((a, b) -> a.getFee() - b.getFee());

Problem 05: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Course POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int findAvgFeeByAdmin(Course[] arr, String admin) {
        int sum = 0, count = 0;
        for (Course c : arr) {
            if (c.getCourseAdmin().equalsIgnoreCase(admin)) {
                sum += c.getFee();
                count++;
            }
        }
        return count > 0 ? (sum / count) : 0;
    }

    public static Course[] sortCourseByFee(Course[] arr, int ratingThreshold) {
        List<Course> list = new ArrayList<>();
        for (Course c : arr) {
            if (c.getRating() <= ratingThreshold) {
                list.add(c);
            }
        }
        if (list.isEmpty()) return null;
        list.sort((a, b) -> a.getFee() - b.getFee());             // ascending by fee
        return list.toArray(new Course[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Course[] courses = new Course[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            String admin = sc.nextLine();
            int fee = sc.nextInt();
            int rating = sc.nextInt();
            sc.nextLine();
            courses[i] = new Course(id, name, admin, fee, rating);
        }
        String targetAdmin = sc.nextLine();
        int targetRating = sc.nextInt();

        int avg = findAvgFeeByAdmin(courses, targetAdmin);
        if (avg > 0) System.out.println(avg);
        else System.out.println("No Such Courses");

        Course[] res = sortCourseByFee(courses, targetRating);
        if (res != null) {
            for (Course c : res) System.out.println(c.getCourseName());
        } else {
            System.out.println("No certificate found");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Course {
    private int courseId;
    private String courseName;
    private String courseAdmin;
    private int fee;
    private int rating;

    public Course(int courseId, String courseName, String courseAdmin, int fee, int rating) {
        this.courseId = courseId;
        this.courseName = courseName;
        this.courseAdmin = courseAdmin;
        this.fee = fee;
        this.rating = rating;
    }

    public int getCourseId() { return courseId; }
    public void setCourseId(int courseId) { this.courseId = courseId; }
    public String getCourseName() { return courseName; }
    public void setCourseName(String courseName) { this.courseName = courseName; }
    public String getCourseAdmin() { return courseAdmin; }
```

```
    public void setCourseAdmin(String courseAdmin) { this.courseAdmin = courseAdmin; }
    public int getFee() { return fee; }
    public void setFee(int fee) { this.fee = fee; }
    public int getRating() { return rating; }
    public void setRating(int rating) { this.rating = rating; }
}
```

Problem 06: Phone Store - Problem Statement and I/O

1. Entity attributes (class Phone)

Class Phone with private attributes: int phoneId, String os, String brand, int price. Constructor plus getters/setters.

2. Static business methods (class Solution)

`findPriceForGivenBrand(Phone[] arr, String brand)`
Returns the TOTAL price of all phones of the given brand (case-insensitive). If none, return 0.

`getPhoneByOsAndMinPrice(Phone[] arr, String os)`
Returns the phone with the given OS (case-insensitive) that has the MINIMUM price. If none, return null.

3. Main execution rules

Read 4 phones (id, os, brand, price), then brand and os. Print the total price (or "The given Brand is not available"). Then print the matching phone's ID (or "The given OS is not available").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - brand and OS not present

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 Android Samsung 20000 2 iOS Apple 70000 3 Android Samsung 15000 4 Android OnePlus 30000 Samsung Android	35000 3	1 Android Samsung 20000 2 iOS Apple 70000 3 Android Samsung 15000 4 Android OnePlus 30000 Nokia Windows	The given Brand is not available The given OS is not available

Explanation: Samsung total = 20000 + 15000 = 35000. Among Android phones the cheapest is phone 3 (15000).

Exam strategy: Min-tracking pattern: Phone min = null; if (min == null || p.getPrice() < min.getPrice()) min = p;

Problem 06: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Phone POJO. Copy as one file, Solution.java.

```
import java.util.Scanner;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int findPriceForGivenBrand(Phone[] arr, String brand) {
        int total = 0;
        for (Phone p : arr) {
            if (p.getBrand().equalsIgnoreCase(brand)) {
                total += p.getPrice();
            }
        }
        return total;
    }

    public static Phone getPhoneByOsAndMinPrice(Phone[] arr, String os) {
        Phone min = null;
        for (Phone p : arr) {
            if (p.getOs().equalsIgnoreCase(os)) {
                if (min == null || p.getPrice() < min.getPrice()) {
                    min = p;
                }
            }
        }
        return min;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Phone[] phones = new Phone[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String os = sc.nextLine();
            String brand = sc.nextLine();
            int price = sc.nextInt();
            sc.nextLine();
            phones[i] = new Phone(id, os, brand, price);
        }
        String searchBrand = sc.nextLine();
        String searchOs = sc.nextLine();

        int total = findPriceForGivenBrand(phones, searchBrand);
        if (total > 0) System.out.println(total);
        else System.out.println("The given Brand is not available");

        Phone res = getPhoneByOsAndMinPrice(phones, searchOs);
        if (res != null) System.out.println(res.getPhoneId());
        else System.out.println("The given OS is not available");
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Phone {
    private int phoneId;
    private String os;
    private String brand;
    private int price;
    public Phone(int phoneId, String os, String brand, int price) {
        this.phoneId = phoneId;
        this.os = os;
        this.brand = brand;
        this.price = price;
    }

    public int getPhoneId() { return phoneId; }
    public void setPhoneId(int phoneId) { this.phoneId = phoneId; }
    public String getOs() { return os; }
    public void setOs(String os) { this.os = os; }
    public String getBrand() { return brand; }
    public void setBrand(String brand) { this.brand = brand; }
    public int getPrice() { return price; }
    public void setPrice(int price) { this.price = price; }
}
```

Problem 07: Book Library - Problem Statement and I/O

1. Entity attributes (class Book)

Class Book with private attributes: int bookId, String title, String author, double price, int pages. Constructor plus getters/setters.

2. Static business methods (class Solution)

`findBookWithMaxPagesByAuthor(Book[] arr, String author)`
Returns the Book with the MAXIMUM pages written by the given author (case-insensitive). If none, return null.

`countBooksAbovePrice(Book[] arr, double threshold)`
Returns the count of books whose price is strictly greater than threshold. If none, return 0.

3. Main execution rules

Read 4 books, then author and price threshold. Print the book's ID and title on separate lines (or "No such book"). Then print the count (or "No books found").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - unknown author, very high threshold

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 JavaBasics Gosling 450.5 300 2 DeepJava Gosling 700 520 3 Python Guido 500 410 4 Algorithms Cormen 900 1200 Gosling 480	2 DeepJava 3	1 JavaBasics Gosling 450.5 300 2 DeepJava Gosling 700 520 3 Python Guido 500 410 4 Algorithms Cormen 900 1200 Knuth 5000	No such book No books found

Explanation: Gosling's book with most pages is DeepJava (520 pages, ID 2). Books priced above 480: DeepJava (700), Python (500), Algorithms (900) = 3.

Exam strategy: Read price with sc.nextDouble() and pages with sc.nextInt(), then flush with sc.nextLine().

Problem 07: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Book POJO. Copy as one file, Solution.java.

```
import java.util.Scanner;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static Book findBookWithMaxPagesByAuthor(Book[] arr, String author) {
        Book maxBook = null;
        for (Book b : arr) {
            if (b.getAuthor().equalsIgnoreCase(author)) {
                if (maxBook == null || b.getPages() > maxBook.getPages()) {
                    maxBook = b;
                }
            }
        }
        return maxBook;
    }

    public static int countBooksAbovePrice(Book[] arr, double threshold) {
        int count = 0;
        for (Book b : arr) {
            if (b.getPrice() > threshold) {
                count++;
            }
        }
        return count;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Book[] books = new Book[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String title = sc.nextLine();
            String author = sc.nextLine();
            double price = sc.nextDouble();
            int pages = sc.nextInt();
            sc.nextLine();
            books[i] = new Book(id, title, author, price, pages);
        }
        String searchAuthor = sc.nextLine();
        double threshold = sc.nextDouble();

        Book res = findBookWithMaxPagesByAuthor(books, searchAuthor);
        if (res != null) {
            System.out.println(res.getBookId());
            System.out.println(res.getTitle());
        } else {
            System.out.println("No such book");
        }

        int count = countBooksAbovePrice(books, threshold);
        if (count > 0) System.out.println(count);
        else System.out.println("No books found");
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Book {
    private int bookId;
    private String title;
    private String author;
    private double price;
    private int pages;

    public Book(int bookId, String title, String author, double price, int pages) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
        this.price = price;
        this.pages = pages;
    }

    public int getBookId() { return bookId; }
    public void setBookId(int bookId) { this.bookId = bookId; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getAuthor() { return author; }
```

```
    public void setAuthor(String author) { this.author = author; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
    public int getPages() { return pages; }
    public void setPages(int pages) { this.pages = pages; }
}
```

Problem 08: Employee Performance - Problem Statement and I/O

1. Entity attributes (class Employee)

Class Employee with private attributes: int empId, String empName, String branch, int rating, double salary. Constructor plus getters/setters.

2. Static business methods (class Solution)

`countEmployeesByBranch(Employee[] arr, String branch)`
Returns the count of employees in the given branch (case-insensitive) with rating >= 4. If none, return 0.

`findEmployeeWithSecondHighestSalary(Employee[] arr)`
Returns the employee with the SECOND HIGHEST distinct salary. If there is no second distinct salary, return null.

3. Main execution rules

Read 4 employees, then branch. Print the count (or "No employee found"). Then print the second-highest-salary employee's ID and name on separate lines (or "No such employee").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - all salaries equal, branch not present

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
201 Ravi IT 5 60000 202 Meena HR 4 45000 203 Kiran IT 3 80000 204 Anil IT 4 70000 IT	2 204 Anil	301 A IT 5 50000 302 B IT 5 50000 303 C HR 4 50000 304 D HR 4 50000 Sales	No employee found No such employee

Explanation: IT employees with rating >= 4: Ravi (5) and Anil (4) = 2. Salaries sorted: 80000 (Kiran), 70000 (Anil) -> second highest is Anil, ID 204.

Exam strategy: Sort a copy DESCENDING, then return the first employee whose salary is strictly less than the top salary. This handles duplicate salaries.

Problem 08: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Employee POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int countEmployeesByBranch(Employee[] arr, String branch) {
        int count = 0;
        for (Employee e : arr) {
            if (e.getBranch().equalsIgnoreCase(branch) && e.getRating() >= 4) {
                count++;
            }
        }
        return count;
    }

    public static Employee findEmployeeWithSecondHighestSalary(Employee[] arr) {
        List<Employee> list = new ArrayList<>(Arrays.asList(arr));
        if (list.isEmpty()) return null;
        list.sort((a, b) -> Double.compare(b.getSalary(), a.getSalary()));  // descending
        double top = list.get(0).getSalary();
        for (Employee e : list) {
            if (e.getSalary() < top) {
                return e;                       // first salary below the maximum
            }
        }
        return null;                            // all salaries are equal
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Employee[] emps = new Employee[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            String branch = sc.nextLine();
            int rating = sc.nextInt();
            double salary = sc.nextDouble();
            sc.nextLine();
            emps[i] = new Employee(id, name, branch, rating, salary);
        }
        String searchBranch = sc.nextLine();

        int count = countEmployeesByBranch(emps, searchBranch);
        if (count > 0) System.out.println(count);
        else System.out.println("No employee found");

        Employee second = findEmployeeWithSecondHighestSalary(emps);
        if (second != null) {
            System.out.println(second.getEmpId());
            System.out.println(second.getEmpName());
        } else {
            System.out.println("No such employee");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Employee {
    private int empId;
    private String empName;
    private String branch;
    private int rating;
    private double salary;

    public Employee(int empId, String empName, String branch, int rating, double salary) {
        this.empId = empId;
        this.empName = empName;
        this.branch = branch;
        this.rating = rating;
        this.salary = salary;
    }

    public int getEmpId() { return empId; }
    public void setEmpId(int empId) { this.empId = empId; }
    public String getEmpName() { return empName; }
    public void setEmpName(String empName) { this.empName = empName; }
    public String getBranch() { return branch; }
```

```
    public void setBranch(String branch) { this.branch = branch; }
    public int getRating() { return rating; }
    public void setRating(int rating) { this.rating = rating; }
    public double getSalary() { return salary; }
    public void setSalary(double salary) { this.salary = salary; }
}
```

Problem 09: Movie Budget Analysis - Problem Statement and I/O

1. Entity attributes (class Movie)

Class Movie with private attributes: String movieName, String company, String genre, int budget, int rating. Constructor plus getters/setters.

2. Static business methods (class Solution)

`findAvgBudgetByGenre(Movie[] arr, String genre)`
Returns the INTEGER average budget of movies of the given genre (case-insensitive). If none, return 0.

`getMoviesByRatingSortedByBudget(Movie[] arr, int rating)`
Returns movies with rating >= the given rating, sorted by budget in DESCENDING order. If none, return null.

3. Main execution rules

Read 4 movies, then genre and rating. Print the average budget (or "No such genre"). Then print the matching movie names one per line (or "No movies found").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - unknown genre, rating 6

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
Inception WB SciFi 160 5 Avatar Fox SciFi 237 4 Titanic Paramount Romance 200 4 Dunkirk WB War 100 3 SciFi 4	198 Avatar Titanic Inception	Inception WB SciFi 160 5 Avatar Fox SciFi 237 4 Titanic Paramount Romance 200 4 Dunkirk WB War 100 3 Horror 6	No such genre No movies found

Explanation: SciFi average = (160 + 237) / 2 = 198 (integer division). Movies with rating >= 4 by budget descending: Avatar (237), Titanic (200), Inception (160).

Exam strategy: If your exam says 'rating equals', change >= to == in the filter; everything else stays the same.

Problem 09: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Movie POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int findAvgBudgetByGenre(Movie[] arr, String genre) {
        int sum = 0, count = 0;
        for (Movie m : arr) {
            if (m.getGenre().equalsIgnoreCase(genre)) {
                sum += m.getBudget();
                count++;
            }
        }
        return count > 0 ? (sum / count) : 0;
    }

    public static Movie[] getMoviesByRatingSortedByBudget(Movie[] arr, int rating) {
        List<Movie> list = new ArrayList<>();
        for (Movie m : arr) {
            if (m.getRating() >= rating) {
                list.add(m);
            }
        }
        if (list.isEmpty()) return null;
        list.sort((a, b) -> b.getBudget() - a.getBudget());       // descending
        return list.toArray(new Movie[0]);
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Movie[] movies = new Movie[4];
        for (int i = 0; i < 4; i++) {
            String name = sc.nextLine();
            String company = sc.nextLine();
            String genre = sc.nextLine();
            int budget = sc.nextInt();
            int rating = sc.nextInt();
            sc.nextLine();
            movies[i] = new Movie(name, company, genre, budget, rating);
        }
        String searchGenre = sc.nextLine();
        int searchRating = sc.nextInt();

        int avg = findAvgBudgetByGenre(movies, searchGenre);
        if (avg > 0) System.out.println(avg);
        else System.out.println("No such genre");

        Movie[] res = getMoviesByRatingSortedByBudget(movies, searchRating);
        if (res != null) {
            for (Movie m : res) System.out.println(m.getMovieName());
        } else {
            System.out.println("No movies found");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Movie {
    private String movieName;
    private String company;
    private String genre;
    private int budget;
    private int rating;

    public Movie(String movieName, String company, String genre, int budget, int rating) {
        this.movieName = movieName;
        this.company = company;
        this.genre = genre;
        this.budget = budget;
        this.rating = rating;
    }

    public String getMovieName() { return movieName; }
    public void setMovieName(String movieName) { this.movieName = movieName; }
    public String getCompany() { return company; }
    public void setCompany(String company) { this.company = company; }
    public String getGenre() { return genre; }
    public void setGenre(String genre) { this.genre = genre; }
```

```
    public int getBudget() { return budget; }
    public void setBudget(int budget) { this.budget = budget; }
    public int getRating() { return rating; }
    public void setRating(int rating) { this.rating = rating; }
}
```

Problem 10: Autonomous Car Testing - Problem Statement and I/O

1. Entity attributes (class AutonomousCar)

Class AutonomousCar with private attributes: int carId, String brand, int testsConducted, int testsPassed, String environment, String grade. The constructor takes only the first FIVE values; grade is initialised to "NA" inside the constructor. Getters/setters for all six.

2. Static business methods (class Solution)

`updateGradeByEnvironment(AutonomousCar[] arr, String env)`

For every car in the given environment (case-insensitive) compute passRate = testsPassed * 100 / testsConducted. Set grade = "A1" if passRate >= 80, else "B2". Return the updated cars; if none, return null.

`findCarWithSecondHighestPassRate(AutonomousCar[] arr)`

Returns the car with the SECOND HIGHEST distinct pass rate (all cars). If there is no second distinct rate, return null.

3. Main execution rules

Read 4 cars (id, brand, tests, passed, environment), then env. Print each updated car's ID and grade on separate lines (or "No car found in the given environment"). Then print the second-highest car's ID and brand (or "No such car").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - unknown environment, all pass rates equal

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 Tesla 100 90 Urban 2 Waymo 200 150 Urban 3 Cruise 50 48 Highway 4 Zoox 80 40 Urban Urban	1 A1 2 B2 4 B2 1 Tesla	1 A 10 5 Urban 2 B 20 10 Urban 3 C 30 15 Urban 4 D 40 20 Urban Rural	No car found in the given environment No such car

Explanation: Urban cars: Tesla 90% -> A1, Waymo 75% -> B2, Zoox 50% -> B2. All pass rates: 90, 75, 96, 50 -> second highest is Tesla (90%), ID 1.

Exam strategy: Use 100.0 (a double) when computing the rate so integer division does not truncate it; guard testsConducted == 0.

Problem 10: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the AutonomousCar POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // helper: pass percentage of one car (guards against divide by zero)
    private static double passRate(AutonomousCar c) {
        if (c.getTestsConducted() == 0) return 0;
        return c.getTestsPassed() * 100.0 / c.getTestsConducted();
    }
    // 1. STATIC BUSINESS METHODS FIRST
    public static AutonomousCar[] updateGradeByEnvironment(AutonomousCar[] arr, String env) {
        List<AutonomousCar> list = new ArrayList<>();
        for (AutonomousCar c : arr) {
            if (c.getEnvironment().equalsIgnoreCase(env)) {
                c.setGrade(passRate(c) >= 80 ? "A1" : "B2");
                list.add(c);
            }
        }
        if (list.isEmpty()) return null;
        return list.toArray(new AutonomousCar[0]);
    }

    public static AutonomousCar findCarWithSecondHighestPassRate(AutonomousCar[] arr) {
        List<AutonomousCar> list = new ArrayList<>(Arrays.asList(arr));
        if (list.isEmpty()) return null;
        list.sort((a, b) -> Double.compare(passRate(b), passRate(a)));   // descending
        double top = passRate(list.get(0));
        for (AutonomousCar c : list) {
            if (passRate(c) < top) {
                return c;
            }
        }
        return null;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AutonomousCar[] cars = new AutonomousCar[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String brand = sc.nextLine();
            int tests = sc.nextInt();
            int passed = sc.nextInt();
            sc.nextLine();
            String env = sc.nextLine();
            cars[i] = new AutonomousCar(id, brand, tests, passed, env);
        }
        String searchEnv = sc.nextLine();

        AutonomousCar[] updated = updateGradeByEnvironment(cars, searchEnv);
        if (updated != null) {
            for (AutonomousCar c : updated) {
                System.out.println(c.getCarId());
                System.out.println(c.getGrade());
            }
        } else {
            System.out.println("No car found in the given environment");
        }

        AutonomousCar second = findCarWithSecondHighestPassRate(cars);
        if (second != null) {
            System.out.println(second.getCarId());
            System.out.println(second.getBrand());
        } else {
            System.out.println("No such car");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class AutonomousCar {
    private int carId;
    private String brand;
    private int testsConducted;
    private int testsPassed;
    private String environment;
```

```
    private String grade;

    public AutonomousCar(int carId, String brand, int testsConducted,
            int testsPassed, String environment) {
        this.carId = carId;
        this.brand = brand;
        this.testsConducted = testsConducted;
        this.testsPassed = testsPassed;
        this.environment = environment;
        this.grade = "NA";
    }

    public int getCarId() { return carId; }
    public void setCarId(int carId) { this.carId = carId; }
    public String getBrand() { return brand; }
    public void setBrand(String brand) { this.brand = brand; }
    public int getTestsConducted() { return testsConducted; }
    public void setTestsConducted(int testsConducted) { this.testsConducted = testsConducted; }
    public int getTestsPassed() { return testsPassed; }
    public void setTestsPassed(int testsPassed) { this.testsPassed = testsPassed; }
    public String getEnvironment() { return environment; }
    public void setEnvironment(String environment) { this.environment = environment; }
    public String getGrade() { return grade; }
    public void setGrade(String grade) { this.grade = grade; }
}
```

Problem 11: Inventory Replenishment - Problem Statement and I/O

1. Entity attributes (class Inventory)

Class Inventory with private attributes: String inventoryId, int maxQty, int curQty, int threshold. Constructor plus getters/setters.

2. Static business methods (class Solution)

`filterItemsNeedingRefill(Inventory[] arr)`

Returns the items whose curQty <= threshold (they need a refill), in input order. If none, return null.

`countByThresholdCategory(Inventory[] arr, String category)`

Category by threshold value: "Critical" if threshold > 75, "Moderate" if 50 to 75 (inclusive), "Normal" if below 50. Returns how many items fall in the given category (case-insensitive). If none, return 0.

3. Main execution rules

Read 4 items (id, max, current, threshold), then category. Print the IDs needing refill one per line (or "No items need refill"). Then print the category count (or "No items in the given category").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - no refills needed, empty category

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
I101 100 20 30 I102 200 150 80 I103 150 60 60 I104 300 90 90 Critical	I101 I103 I104 2	I201 100 90 30 I202 100 80 20 I203 100 70 10 I204 100 60 40 Critical	No items need refill No items in the given category

Explanation: Refill needed (cur <= threshold): I101 (20<=30), I103 (60<=60), I104 (90<=90). Critical (threshold > 75): I102 (80) and I104 (90) = 2.

Exam strategy: Note the <= : an item exactly AT its threshold also needs a refill (I103 and I104 in the sample).

Problem 11: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Inventory POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // helper: classify a threshold value
    private static String categoryOf(int threshold) {
        if (threshold > 75) return "Critical";
        if (threshold >= 50) return "Moderate";
        return "Normal";
    }
    // 1. STATIC BUSINESS METHODS FIRST
    public static Inventory[] filterItemsNeedingRefill(Inventory[] arr) {
        List<Inventory> list = new ArrayList<>();
        for (Inventory inv : arr) {
            if (inv.getCurQty() <= inv.getThreshold()) {
                list.add(inv);
            }
        }
        if (list.isEmpty()) return null;
        return list.toArray(new Inventory[0]);
    }

    public static int countByThresholdCategory(Inventory[] arr, String category) {
        int count = 0;
        for (Inventory inv : arr) {
            if (categoryOf(inv.getThreshold()).equalsIgnoreCase(category)) {
                count++;
            }
        }
        return count;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Inventory[] items = new Inventory[4];
        for (int i = 0; i < 4; i++) {
            String id = sc.nextLine();
            int max = sc.nextInt();
            int cur = sc.nextInt();
            int thr = sc.nextInt();
            sc.nextLine();
            items[i] = new Inventory(id, max, cur, thr);
        }
        String category = sc.nextLine();

        Inventory[] refill = filterItemsNeedingRefill(items);
        if (refill != null) {
            for (Inventory inv : refill) System.out.println(inv.getInventoryId());
        } else {
            System.out.println("No items need refill");
        }

        int count = countByThresholdCategory(items, category);
        if (count > 0) System.out.println(count);
        else System.out.println("No items in the given category");
        sc.close();
    }
}
// 3. ENTITY POJO CLASS
class Inventory {
    private String inventoryId;
    private int maxQty;
    private int curQty;
    private int threshold;

    public Inventory(String inventoryId, int maxQty, int curQty, int threshold) {
        this.inventoryId = inventoryId;
        this.maxQty = maxQty;
        this.curQty = curQty;
        this.threshold = threshold;
    }

    public String getInventoryId() { return inventoryId; }
    public void setInventoryId(String inventoryId) { this.inventoryId = inventoryId; }
    public int getMaxQty() { return maxQty; }
    public void setMaxQty(int maxQty) { this.maxQty = maxQty; }
    public int getCurQty() { return curQty; }
```

```
    public void setCurQty(int curQty) { this.curQty = curQty; }
    public int getThreshold() { return threshold; }
    public void setThreshold(int threshold) { this.threshold = threshold; }
}
```

Problem 12: Hotel Booking - Problem Statement and I/O

1. Entity attributes (class Hotel)

Class Hotel with private attributes: int hotelId, String hotelName, String bookingDate (format dd-MM-yyyy), int roomsBooked, String wifi, double totalBill. Constructor plus getters/setters.

2. Static business methods (class Solution)

`noOfRoomsBookedInGivenMonth(Hotel[] arr, String month)`
Returns the TOTAL rooms booked in the given month. The month may be a full name (March) or 3-letter short form (Mar), case-insensitive. If none, return 0.

`searchHotelWithSecondHighestBill(Hotel[] arr)`
Returns the hotel with the SECOND HIGHEST distinct totalBill. If there is no second distinct bill, return null.

3. Main execution rules

Read 4 hotels, then month. Print the total rooms (or "No rooms booked in the given month"). Then print the second-highest-bill hotel's ID and name on separate lines (or "No such hotel").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - no bookings in December, all bills equal

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 Taj 05-03-2023 3 yes 12000 2 Oberoi 15-03-2023 2 no 18000.5 3 Hyatt 20-04-2023 4 yes 15000 4 ITC 25-03-2023 1 yes 9000 March	6 3 Hyatt	1 Taj 05-03-2023 3 yes 10000 2 Oberoi 15-03-2023 2 no 10000 3 Hyatt 20-04-2023 4 yes 10000 4 ITC 25-03-2023 1 yes 10000 December	No rooms booked in the given month No such hotel

Explanation: March bookings: Taj (3) + Oberoi (2) + ITC (1) = 6 rooms. Bills sorted: 18000.5 (Oberoi), 15000 (Hyatt) -> second highest is Hyatt, ID 3.

Exam strategy: Parse the date with LocalDate.parse(date, DateTimeFormatter.ofPattern("dd-MM-yyyy")) and compare getMonth().name().

Problem 12: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Hotel POJO. Copy as one file, Solution.java.

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.*;

public class Solution {

    // helper: does the booking date fall in the given month (Mar or March)?
    private static boolean isInMonth(String date, String month) {
        try {
            LocalDate d = LocalDate.parse(date, DateTimeFormatter.ofPattern("dd-MM-yyyy"));
            String full = d.getMonth().name();                      // e.g. MARCH
            return full.equalsIgnoreCase(month)
                || full.substring(0, 3).equalsIgnoreCase(month);    // MAR
        } catch (DateTimeParseException e) {
            return false;
        }
    }

    // 1. STATIC BUSINESS METHODS FIRST
    public static int noOfRoomsBookedInGivenMonth(Hotel[] arr, String month) {
        int total = 0;
        for (Hotel h : arr) {
            if (isInMonth(h.getBookingDate(), month)) {
                total += h.getRoomsBooked();
            }
        }
        return total;
    }
    public static Hotel searchHotelWithSecondHighestBill(Hotel[] arr) {
        List<Hotel> list = new ArrayList<>(Arrays.asList(arr));
        if (list.isEmpty()) return null;
        list.sort((a, b) -> Double.compare(b.getTotalBill(), a.getTotalBill()));  // descending
        double top = list.get(0).getTotalBill();
        for (Hotel h : list) {
            if (h.getTotalBill() < top) {
                return h;
            }
        }
        return null;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Hotel[] hotels = new Hotel[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            String date = sc.nextLine();
            int rooms = sc.nextInt();
            sc.nextLine();
            String wifi = sc.nextLine();
            double bill = sc.nextDouble();
            sc.nextLine();
            hotels[i] = new Hotel(id, name, date, rooms, wifi, bill);
        }
        String month = sc.nextLine();
        int rooms = noOfRoomsBookedInGivenMonth(hotels, month);
        if (rooms > 0) System.out.println(rooms);
        else System.out.println("No rooms booked in the given month");

        Hotel second = searchHotelWithSecondHighestBill(hotels);
        if (second != null) {
            System.out.println(second.getHotelId());
            System.out.println(second.getHotelName());
        } else {
            System.out.println("No such hotel");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Hotel {
    private int hotelId;
    private String hotelName;
```

```
    private String bookingDate;
    private int roomsBooked;
    private String wifi;
    private double totalBill;

    public Hotel(int hotelId, String hotelName, String bookingDate, int roomsBooked,
            String wifi, double totalBill) {
        this.hotelId = hotelId;
        this.hotelName = hotelName;
        this.bookingDate = bookingDate;
        this.roomsBooked = roomsBooked;
        this.wifi = wifi;
        this.totalBill = totalBill;
    }

    public int getHotelId() { return hotelId; }
    public void setHotelId(int hotelId) { this.hotelId = hotelId; }
    public String getHotelName() { return hotelName; }
    public void setHotelName(String hotelName) { this.hotelName = hotelName; }
    public String getBookingDate() { return bookingDate; }
    public void setBookingDate(String bookingDate) { this.bookingDate = bookingDate; }
    public int getRoomsBooked() { return roomsBooked; }
    public void setRoomsBooked(int roomsBooked) { this.roomsBooked = roomsBooked; }
    public String getWifi() { return wifi; }
    public void setWifi(String wifi) { this.wifi = wifi; }
    public double getTotalBill() { return totalBill; }
    public void setTotalBill(double totalBill) { this.totalBill = totalBill; }
}
```

Problem 13: Naval Vessel Fleet - Problem Statement and I/O

1. Entity attributes (class NavalVessel)

Class NavalVessel with private attributes: int vesselId, String vesselName, int voyages, String mode, String vesselClass. Constructor plus getters/setters.

2. Static business methods (class Solution)

`findAvgVoyagesByVesselClass(NavalVessel[] arr, String vesselClass)`
Returns the INTEGER average voyages of vessels of the given class (case-insensitive). If none, return 0.

`findVesselWithHighestVoyagesByMode(NavalVessel[] arr, String mode)`
Returns the vessel with the HIGHEST voyages among vessels with the given mode (case-insensitive). If none, return null.

3. Main execution rules

Read 4 vessels, then vessel class and mode. Print the average voyages (or "No such vessel class"). Then print the vessel's ID and name on separate lines (or "No such mode").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - unknown class and mode

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 Vikrant 40 Combat Carrier 2 Kolkata 30 Combat Destroyer 3 Sandhayak 25 Survey Support 4 Vikramaditya 50 Combat Carrier Carrier Combat	45 4 Vikramaditya	1 Vikrant 40 Combat Carrier 2 Kolkata 30 Combat Destroyer 3 Sandhayak 25 Survey Support 4 Vikramaditya 50 Combat Carrier Submarine Rescue	No such vessel class No such mode

Explanation: Carrier average = (40 + 50) / 2 = 45. In Combat mode the highest voyages is Vikramaditya (50), ID 4.

Exam strategy: Same 'max with filter' skeleton as Problems 1 and 7: null check first, then compare.

Problem 13: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the NavalVessel POJO. Copy as one file, Solution.java.

```
import java.util.Scanner;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int findAvgVoyagesByVesselClass(NavalVessel[] arr, String vesselClass) {
        int sum = 0, count = 0;
        for (NavalVessel v : arr) {
            if (v.getVesselClass().equalsIgnoreCase(vesselClass)) {
                sum += v.getVoyages();
                count++;
            }
        }
        return count > 0 ? (sum / count) : 0;
    }

    public static NavalVessel findVesselWithHighestVoyagesByMode(NavalVessel[] arr, String mode) {
        NavalVessel max = null;
        for (NavalVessel v : arr) {
            if (v.getMode().equalsIgnoreCase(mode)) {
                if (max == null || v.getVoyages() > max.getVoyages()) {
                    max = v;
                }
            }
        }
        return max;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        NavalVessel[] vessels = new NavalVessel[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            int voyages = sc.nextInt();
            sc.nextLine();
            String mode = sc.nextLine();
            String cls = sc.nextLine();
            vessels[i] = new NavalVessel(id, name, voyages, mode, cls);
        }
        String searchClass = sc.nextLine();
        String searchMode = sc.nextLine();

        int avg = findAvgVoyagesByVesselClass(vessels, searchClass);
        if (avg > 0) System.out.println(avg);
        else System.out.println("No such vessel class");

        NavalVessel res = findVesselWithHighestVoyagesByMode(vessels, searchMode);
        if (res != null) {
            System.out.println(res.getVesselId());
            System.out.println(res.getVesselName());
        } else {
            System.out.println("No such mode");
        }
        sc.close();
    }
}
// 3. ENTITY POJO CLASS
class NavalVessel {
    private int vesselId;
    private String vesselName;
    private int voyages;
    private String mode;
    private String vesselClass;

    public NavalVessel(int vesselId, String vesselName, int voyages,
            String mode, String vesselClass) {
        this.vesselId = vesselId;
        this.vesselName = vesselName;
        this.voyages = voyages;
        this.mode = mode;
        this.vesselClass = vesselClass;
    }

    public int getVesselId() { return vesselId; }
    public void setVesselId(int vesselId) { this.vesselId = vesselId; }
    public String getVesselName() { return vesselName; }
```

```
    public void setVesselName(String vesselName) { this.vesselName = vesselName; }
    public int getVoyages() { return voyages; }
    public void setVoyages(int voyages) { this.voyages = voyages; }
    public String getMode() { return mode; }
    public void setMode(String mode) { this.mode = mode; }
    public String getVesselClass() { return vesselClass; }
    public void setVesselClass(String vesselClass) { this.vesselClass = vesselClass; }
}
```

Problem 14: SIM Card Management - Problem Statement and I/O

1. Entity attributes (class Sim)

Class Sim with private attributes: int simId, String customerName, double balance, double ratePerSecond, String circle. Constructor plus getters/setters.

2. Static business methods (class Solution)

`transferBalance(Sim[] arr, int fromId, int toId, double amount)`
Moves amount from one SIM to another. Returns true only if both SIMs exist, they are different and the sender has balance >= amount; otherwise return false and change nothing.

`changeCircle(Sim[] arr, int simId, String newCircle)`
Updates the circle of the SIM with the given ID. Returns true if found, else false.

`sortSimsByBalanceDesc(Sim[] arr)`
Returns a NEW array of all SIMs sorted by balance in DESCENDING order (the original array is left untouched).

3. Main execution rules

Read 4 SIMs, then fromId, toId, amount, then simId and newCircle. Print "Transfer successful" or "Transfer failed". Print "Circle updated" or "Sim not found". Finally print each SIM from the sorted array as: id name balance.

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - receiver and SIM id do not exist

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 Amit 500.0 0.5 Kolkata 2 Raj 300.0 0.4 Delhi 3 Sita 800.0 0.6 Mumbai 4 Neha 150.0 0.3 Delhi 3 4 200 2 Chennai	Transfer successful Circle updated 3 Sita 600.0 1 Amit 500.0 4 Neha 350.0 2 Raj 300.0	1 Amit 500.0 0.5 Kolkata 2 Raj 300.0 0.4 Delhi 3 Sita 800.0 0.6 Mumbai 4 Neha 150.0 0.3 Delhi 4 9 50 7 Chennai	Transfer failed Sim not found 3 Sita 800.0 1 Amit 500.0 2 Raj 300.0 4 Neha 150.0

Explanation: Sita (800) sends 200 to Neha (150): balances become 600 and 350. Raj's circle changes to Chennai. Sorted by balance: Sita 600, Amit 500, Neha 350, Raj 300.

Exam strategy: Check every failure condition BEFORE changing any balance, so a failed transfer never leaves half-updated data.

Problem 14: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Sim POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // helper: find a SIM by id (null if missing)
    private static Sim findById(Sim[] arr, int id) {
        for (Sim s : arr) {
            if (s.getSimId() == id) return s;
        }
        return null;
    }

    // 1. STATIC BUSINESS METHODS FIRST
    public static boolean transferBalance(Sim[] arr, int fromId, int toId, double amount) {
        Sim from = findById(arr, fromId);
        Sim to = findById(arr, toId);
        if (from == null || to == null || from == to) return false;
        if (amount <= 0 || from.getBalance() < amount) return false;
        from.setBalance(from.getBalance() - amount);
        to.setBalance(to.getBalance() + amount);
        return true;
    }

    public static boolean changeCircle(Sim[] arr, int simId, String newCircle) {
        Sim s = findById(arr, simId);
        if (s == null) return false;
        s.setCircle(newCircle);
        return true;
    }
    public static Sim[] sortSimsByBalanceDesc(Sim[] arr) {
        Sim[] copy = Arrays.copyOf(arr, arr.length);
        Arrays.sort(copy, (a, b) -> Double.compare(b.getBalance(), a.getBalance()));
        return copy;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Sim[] sims = new Sim[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            double balance = sc.nextDouble();
            double rate = sc.nextDouble();
            sc.nextLine();
            String circle = sc.nextLine();
            sims[i] = new Sim(id, name, balance, rate, circle);
        }
        int fromId = sc.nextInt();
        int toId = sc.nextInt();
        double amount = sc.nextDouble();
        int changeId = sc.nextInt();
        sc.nextLine();
        String newCircle = sc.nextLine();

        if (transferBalance(sims, fromId, toId, amount)) System.out.println("Transfer successful");
        else System.out.println("Transfer failed");
        if (changeCircle(sims, changeId, newCircle)) System.out.println("Circle updated");
        else System.out.println("Sim not found");

        for (Sim s : sortSimsByBalanceDesc(sims)) {
            System.out.println(s.getSimId() + " " + s.getCustomerName() + " " + s.getBalance());
        }
        sc.close();
    }
}
// 3. ENTITY POJO CLASS
class Sim {
    private int simId;
    private String customerName;
    private double balance;
    private double ratePerSecond;
    private String circle;

    public Sim(int simId, String customerName, double balance,
            double ratePerSecond, String circle) {
```

```
        this.simId = simId;
        this.customerName = customerName;
        this.balance = balance;
        this.ratePerSecond = ratePerSecond;
        this.circle = circle;
    }

    public int getSimId() { return simId; }
    public void setSimId(int simId) { this.simId = simId; }
    public String getCustomerName() { return customerName; }
    public void setCustomerName(String customerName) { this.customerName = customerName; }
    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }
    public double getRatePerSecond() { return ratePerSecond; }
    public void setRatePerSecond(double ratePerSecond) { this.ratePerSecond = ratePerSecond; }
    public String getCircle() { return circle; }
    public void setCircle(String circle) { this.circle = circle; }
}
```

Problem 15: Footwear Store - Problem Statement and I/O

1. Entity attributes (class Footwear)

Class Footwear with private attributes: int footwearId, String footwearName, String footwearType, int price. Constructor plus getters/setters.

2. Static business methods (class Solution)

`countFootwearByType(Footwear[] arr, String type)`
Returns the count of footwear of the given type (case-insensitive). If none, return 0.

`getSecondHighestPriceByName(Footwear[] arr, String name)`
Among footwear with the given name (case-insensitive) returns the one with the SECOND HIGHEST distinct price. If there is no second distinct price, return null.

3. Main execution rules

Read 4 footwear items, then type and name. Print the count (or "No footwear of the given type"). Then print the footwear's ID and price on separate lines (or "No such footwear").

Test runs (real output of the code on the next page) - left: sample from the question; right: edge case - unknown type, only one Bata item

SAMPLE INPUT	SAMPLE OUTPUT	EDGE-CASE INPUT	EDGE-CASE OUTPUT
1 Nike Sports 5000 2 Bata Formal 2000 3 Nike Casual 4000 4 Nike Sports 6000 Sports Nike	2 1 5000	1 Nike Sports 5000 2 Bata Formal 2000 3 Nike Casual 4000 4 Nike Sports 6000 Sandals Bata	No footwear of the given type No such footwear

Explanation: Two items are of type Sports (IDs 1 and 4). Nike prices are 6000, 5000, 4000, so the second highest is 5000 (ID 1).

Exam strategy: Filter by name FIRST into a list, then sort that list descending and skip entries equal to the top price.

Problem 15: Complete Uncut Java Implementation

Complete file: Solution class (business methods + main) followed by the Footwear POJO. Copy as one file, Solution.java.

```
import java.util.*;

public class Solution {

    // 1. STATIC BUSINESS METHODS FIRST
    public static int countFootwearByType(Footwear[] arr, String type) {
        int count = 0;
        for (Footwear f : arr) {
            if (f.getFootwearType().equalsIgnoreCase(type)) {
                count++;
            }
        }
        return count;
    }

    public static Footwear getSecondHighestPriceByName(Footwear[] arr, String name) {
        List<Footwear> list = new ArrayList<>();
        for (Footwear f : arr) {
            if (f.getFootwearName().equalsIgnoreCase(name)) {
                list.add(f);
            }
        }
        if (list.isEmpty()) return null;
        list.sort((a, b) -> b.getPrice() - a.getPrice());          // descending
        int top = list.get(0).getPrice();
        for (Footwear f : list) {
            if (f.getPrice() < top) {
                return f;
            }
        }
        return null;
    }

    // 2. MAIN EXECUTION RUNNER
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Footwear[] items = new Footwear[4];
        for (int i = 0; i < 4; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            String type = sc.nextLine();
            int price = sc.nextInt();
            sc.nextLine();
            items[i] = new Footwear(id, name, type, price);
        }
        String searchType = sc.nextLine();
        String searchName = sc.nextLine();

        int count = countFootwearByType(items, searchType);
        if (count > 0) System.out.println(count);
        else System.out.println("No footwear of the given type");

        Footwear res = getSecondHighestPriceByName(items, searchName);
        if (res != null) {
            System.out.println(res.getFootwearId());
            System.out.println(res.getPrice());
        } else {
            System.out.println("No such footwear");
        }
        sc.close();
    }
}

// 3. ENTITY POJO CLASS
class Footwear {
    private int footwearId;
    private String footwearName;
    private String footwearType;
    private int price;
    public Footwear(int footwearId, String footwearName, String footwearType, int price) {
        this.footwearId = footwearId;
        this.footwearName = footwearName;
        this.footwearType = footwearType;
        this.price = price;
    }

    public int getFootwearId() { return footwearId; }
    public void setFootwearId(int footwearId) { this.footwearId = footwearId; }
```

```
    public String getFootwearName() { return footwearName; }
    public void setFootwearName(String footwearName) { this.footwearName = footwearName; }
    public String getFootwearType() { return footwearType; }
    public void setFootwearType(String footwearType) { this.footwearType = footwearType; }
    public int getPrice() { return price; }
    public void setPrice(int price) { this.price = price; }
}
```